Java Quiz – Ausführliche Analyse der 31 Fragen

Benjamin Malicke ©

26. Oktober 2025

Inhaltsverzeichnis

1 Einleitung **4**

2 Frage 0 **5**
2.1 Code . . . 5
2.2 Initialisierung . . . 5
2.3 Schritt-für-Schritt-Ausführung . . . 5
2.4 Ausgabe . . . 5
2.5 Zusätzliche Erklärungen . . . 5

3 Frage 1 **5**
3.1 Code . . . 5
3.2 Initialisierung . . . 5
3.3 Schritt-für-Schritt-Ausführung . . . 5
3.4 Ausgabe . . . 6
3.5 Zusätzliche Erklärungen . . . 6

4 Frage 2 **6**
4.1 Code . . . 6
4.2 Initialisierung . . . 6
4.3 Schritt-für-Schritt-Ausführung . . . 6
4.4 Ausgabe . . . 6
4.5 Zusätzliche Erklärungen . . . 6

5 Frage 3 **6**
5.1 Code . . . 6
5.2 Initialisierung . . . 6
5.3 Schritt-für-Schritt-Ausführung . . . 6
5.4 Ausgabe . . . 7
5.5 Zusätzliche Erklärungen . . . 7

6 Frage 4 **7**
6.1 Code . . . 7
6.2 Initialisierung . . . 7
6.3 Schritt-für-Schritt-Ausführung . . . 7
6.4 Ausgabe . . . 7
6.5 Zusätzliche Erklärungen . . . 7

7 Frage 5 **7**
7.1 Code . . . 7
7.2 Initialisierung . . . 7
7.3 Schritt-für-Schritt-Ausführung . . . 7
7.4 Ausgabe . . . 8
7.5 Zusätzliche Erklärungen . . . 8

8 Frage 6 **8**
8.1 Code . . . 8
8.2 Initialisierung . . . 8
8.3 Schritt-für-Schritt-Ausführung . . . 8
8.4 Ausgabe . . . 8
8.5 Zusätzliche Erklärungen . . . 8

9 Frage 7 **8**
9.1 Code . . . 8
9.2 Initialisierung . . . 8
9.3 Schritt-für-Schritt-Ausführung . . . 9
9.4 Ausgabe . . . 9
9.5 Zusätzliche Erklärungen . . . 9

10 Frage 8 **9**
10.1 Code . . . 9
10.2 Initialisierung . . . 9
10.3 Schritt-für-Schritt-Ausführung . . . 9
10.4 Ausgabe . . . 9
10.5 Zusätzliche Erklärungen . . . 9

11 Frage 9 **9**
11.1 Code . . . 9
11.2 Initialisierung . . . 10
11.3 Schritt-für-Schritt-Ausführung . . . 10
11.4 Ausgabe . . . 10
11.5 Zusätzliche Erklärungen . . . 10

12 Frage 10 **10**
12.1 Code . . . 10
12.2 Initialisierung . . . 10
12.3 Schritt-für-Schritt-Ausführung . . . 10
12.4 Ausgabe . . . 10
12.5 Zusätzliche Erklärungen . . . 10

13 Frage 11 **10**
13.1 Code . . . 10
13.2 Initialisierung . . . 11
13.3 Schritt-für-Schritt-Ausführung . . . 11
13.4 Ausgabe . . . 11
13.5 Zusätzliche Erklärungen . . . 11

14 Frage 12 **11**
14.1 Code . . . 11
14.2 Initialisierung . . . 11
14.3 Schritt-für-Schritt-Ausführung . . . 11
14.4 Ausgabe . . . 11
14.5 Zusätzliche Erklärungen . . . 11

15 Frage 13 **11**
15.1 Code . . . 11
15.2 Initialisierung . . . 12
15.3 Schritt-für-Schritt-Ausführung . . . 12
15.4 Ausgabe . . . 12
15.5 Zusätzliche Erklärungen . . . 12

16 Frage 14 **12**
16.1 Code . . . 12
16.2 Initialisierung . . . 12
16.3 Schritt-für-Schritt-Ausführung . . . 12
16.4 Ausgabe . . . 12
16.5 Zusätzliche Erklärungen . . . 12

17 Frage 15 **12**
17.1 Code . . . 12
17.2 Initialisierung . . . 13
17.3 Schritt-für-Schritt-Ausführung . . . 13
17.4 Ausgabe . . . 13
17.5 Zusätzliche Erklärungen . . . 13

18 Frage 16 **13**
18.1 Code . . . 13
18.2 Initialisierung . . . 13
18.3 Schritt-für-Schritt-Ausführung . . . 13
18.4 Ausgabe . . . 13
18.5 Zusätzliche Erklärungen . . . 13

19 Frage 17 **13**
19.1 Code . . . 13
19.2 Initialisierung . . . 14
19.3 Schritt-für-Schritt-Ausführung . . . 14
19.4 Ausgabe . . . 14
19.5 Zusätzliche Erklärungen . . . 14

20 Frage 18 **14**
20.1 Code . . . 14
20.2 Initialisierung . . . 14
20.3 Schritt-für-Schritt-Ausführung . . . 14
20.4 Ausgabe . . . 14
20.5 Zusätzliche Erklärungen . . . 14

21 Frage 19 **14**
21.1 Code . . . 14
21.2 Initialisierung . . . 15
21.3 Schritt-für-Schritt-Ausführung . . . 15
21.4 Ausgabe . . . 15
21.5 Zusätzliche Erklärungen . . . 15

22 Frage 20 **15**
22.1 Code . . . 15
22.2 Initialisierung . . . 15
22.3 Schritt-für-Schritt-Ausführung . . . 15
22.4 Ausgabe . . . 15
22.5 Zusätzliche Erklärungen . . . 15

23 Frage 21 **15**
23.1 Code . . . 15
23.2 Initialisierung . . . 15
23.3 Schritt-für-Schritt-Ausführung . . . 16
23.4 Ausgabe . . . 16
23.5 Zusätzliche Erklärungen . . . 16

24 Frage 22 **16**
24.1 Code . . . 16
24.2 Initialisierung . . . 16
24.3 Schritt-für-Schritt-Ausführung . . . 16
24.4 Ausgabe . . . 16
24.5 Zusätzliche Erklärungen . . . 16

25 Frage 23 **16**
25.1 Code . . . 16
25.2 Initialisierung . . . 16
25.3 Schritt-für-Schritt-Ausführung . . . 17
25.4 Ausgabe . . . 17
25.5 Zusätzliche Erklärungen . . . 17

26 Frage 24 **17**
26.1 Code . . . 17
26.2 Initialisierung . . . 17
26.3 Schritt-für-Schritt-Ausführung . . . 17
26.4 Ausgabe . . . 17
26.5 Zusätzliche Erklärungen . . . 17

27 Frage 25 **17**
27.1 Code . . . 17
27.2 Initialisierung . . . 17
27.3 Schritt-für-Schritt-Ausführung . . . 18
27.4 Ausgabe . . . 18
27.5 Zusätzliche Erklärungen . . . 18

28 Frage 26 **18**
28.1 Code . . . 18
28.2 Initialisierung . . . 18
28.3 Schritt-für-Schritt-Ausführung . . . 18
28.4 Ausgabe . . . 18
28.5 Zusätzliche Erklärungen . . . 18

29 Frage 27 **18**
29.1 Code . . . 18
29.2 Initialisierung . . . 18
29.3 Schritt-für-Schritt-Ausführung . . . 19
29.4 Ausgabe . . . 19
29.5 Zusätzliche Erklärungen . . . 19

30 Frage 28 **19**
30.1 Code . . . 19
30.2 Initialisierung . . . 19
30.3 Schritt-für-Schritt-Ausführung . . . 19
30.4 Ausgabe . . . 19
30.5 Zusätzliche Erklärungen . . . 19

31 Frage 29 **19**
31.1 Code . . . 19
31.2 Initialisierung . . . 19
31.3 Schritt-für-Schritt-Ausführung . . . 20
31.4 Ausgabe . . . 20
31.5 Zusätzliche Erklärungen . . . 20

32 Frage 30 **20**
32.1 Code . . . 20
32.2 Initialisierung . . . 20
32.3 Schritt-für-Schritt-Ausführung . . . 20
32.4 Ausgabe . . . 20
32.5 Zusätzliche Erklärungen . . . 20

1 Einleitung

Dieses Dokument analysiert die 31 Fragen des Java Quiz. Jede Frage besteht aus einem Java-Code-Snippet, das Schleifen, Inkrement/Dekrement-Operatoren, bedingte Anweisungen, `break`, `continue` oder ternäre Operatoren verwendet. Wir präsentieren den Code, führen eine schrittweise Ausführung (Dry-Run) durch und erklären die Ergebnisse ausführlich auf Deutsch. Ziel ist es, ein tiefes Verständnis der Konzepte zu fördern, insbesondere der Reihenfolge von Inkrement/Dekrement-Operationen und der Steuerung des Programmflusses.

Die Fragen sind von 0 bis 30 nummeriert. Für jede Frage:

- **Code:** Der vollständige Java-Code.
- **Initialisierung:** Beschreibung des anfänglichen Zustands.

- **Schritt-für-Schritt-Ausführung:** Detaillierte Analyse jeder Operation.
- **Ausgabe:** Das Endergebnis und warum es so ist.
- **Zusätzliche Erklärungen:** Hinweise zu Schleifen, Inkrement/Dekrement, `break`/`continue` und ternären Operatoren.

2 Frage 0

2.1 Code

```
int x = 7;
for (int i = 1; i <= 3; i++) {
  x -= i++ / 2;
  System.out.print(x);
}
```

2.2 Initialisierung

x=7, i=1.

2.3 Schritt-für-Schritt-Ausführung

1. 1. Durchlauf: i=1, i++ → nutzt 1, i=2, 1/2=0, x=7-0=7, print 7, i++ → i=3.
2. 2. Durchlauf: i=3, i++ → nutzt 3, i=4, 3/2=1, x=7-1=6, print 6, i++ → i=5.
3. $i = 5 \leq 3 \rightarrow$ falsch, Ende.

2.4 Ausgabe

Ausgabe: 76

2.5 Zusätzliche Erklärungen

Der Post-Inkrement `i++` im Ausdruck `i++/2` verwendet den aktuellen Wert von `i`, bevor er inkrementiert wird. Integer-Division (/) rundet ab. Die Schleife läuft nur zweimal, da `i` im Schleifenkopf erneut inkrementiert wird.

3 Frage 1

3.1 Code

```
int x = 3;
while (x++ <= 5) {
  System.out.print(x / 2);
}
```

3.2 Initialisierung

x=3.

3.3 Schritt-für-Schritt-Ausführung

1. x++=3, x=4, 3≤5 → print 4/2=2.
2. x++=4, x=5, 4≤5 → print 5/2=2.
3. x++=5, x=6, 5≤5 → print 6/2=3.
4. x=6, 6≤5 → falsch, Ende.

3.4 Ausgabe

Ausgabe: 223

3.5 Zusätzliche Erklärungen

`x++` in der Bedingung verwendet den aktuellen Wert von `x`, inkrementiert aber `x` danach. Die Ausgabe basiert auf dem inkrementierten Wert von `x`.

4 Frage 2

4.1 Code

```
int x = 4, y = 5;
for (int i = 0; i < 2; i++) {
  x += y-- / 2;
}
System.out.print(x + ":" + y);
```

4.2 Initialisierung

x=4, y=5, i=0.

4.3 Schritt-für-Schritt-Ausführung

1. i=0: y--=5, y=4, 5/2=2, x=4+2=6.
2. i=1: y--=4, y=3, 4/2=2, x=6+2=8.
3. print 8:3.

4.4 Ausgabe

Ausgabe: 8:3

4.5 Zusätzliche Erklärungen

`y--` verwendet den aktuellen Wert von `y`, dekrementiert danach. Integer-Division rundet ab.

5 Frage 3

5.1 Code

```
int i = 0;
while (i++ < 4) {
  if (i == 3) continue;
  System.out.print(i / 2);
}
```

5.2 Initialisierung

i=0.

5.3 Schritt-für-Schritt-Ausführung

1. i++=0, i=1, 0<4, i=1, print 1/2=0.
2. i++=1, i=2, 1<4, i=2, print 2/2=1.
3. i++=2, i=3, 2<4, i=3, continue.
4. i++=3, i=4, 3<4, i=4, print 4/2=2.
5. i++=4, i=5, 4<4 → falsch, Ende.

5.4 Ausgabe

Ausgabe: 012

5.5 Zusätzliche Erklärungen

`continue` überspringt die Ausgabe bei i=3. `i++` in der Bedingung inkrementiert vor der Prüfung.

6 Frage 4

6.1 Code

```
int x = 5;
int y = (x++ / 2 >= 2) ? x-- : ++x;
System.out.print(y);
```

6.2 Initialisierung

x=5.

6.3 Schritt-für-Schritt-Ausführung

1. x++=5, x=6, 5/2=2, 2≥2 → wahr, y=x--=6, x=5.
2. print y=6.

6.4 Ausgabe

Ausgabe: 6

6.5 Zusätzliche Erklärungen

Ternärer Operator: Bedingung `x++/2 >= 2` verwendet `x`=5, inkrementiert dann `x`. Da die Bedingung wahr ist, wird `x--` ausgeführt.

7 Frage 5

7.1 Code

```
for (int i = 1; i <= 3; i++) {
  for (int j = 1; j <= i; j++) {
    if (j / 2 == 1) break;
    System.out.print(j);
  }
  System.out.println();
}
```

7.2 Initialisierung

i=1.

7.3 Schritt-für-Schritt-Ausführung

1. i=1: j=1, 1/2=0 ≠ 1, print 1, `println()`.
2. i=2: j=1, print 1; j=2, 2/2=1, break, `println()`.
3. i=3: j=1, print 1; j=2, break, `println()`.

7.4 Ausgabe

Ausgabe:

```
1
1
1
```

7.5 Zusätzliche Erklärungen

Die innere Schleife bricht bei `j/2==1` (j=2). `println()` fügt nach jeder äußeren Schleife eine neue Zeile hinzu.

8 Frage 6

8.1 Code

```java
int x = 4;
do {
  System.out.print(x / 2);
  x--;
} while (x-- > 2);
```

8.2 Initialisierung

x=4.

8.3 Schritt-für-Schritt-Ausführung

1. x=4, print 4/2=2, x=3.
2. x--=3, x=2, 3>2 → wahr.
3. x=2, print 2/2=1, x=1.
4. x--=1, x=0, 1>2 → falsch, Ende.

8.4 Ausgabe

Ausgabe: 21

8.5 Zusätzliche Erklärungen

Do-while-Schleife führt mindestens einmal aus. `x--` in der Bedingung verwendet den aktuellen Wert, dekrementiert danach.

9 Frage 7

9.1 Code

```java
int sum = 0;
for (int i = 1; i <= 5; i++) {
  if (i / 2 == 1) continue;
  sum += i;
}
System.out.print(sum);
```

9.2 Initialisierung

sum=0, i=1.

9.3 Schritt-für-Schritt-Ausführung

1. i=1: $1/2=0 \neq 1$, sum=0+1=1.
2. i=2: 2/2=1, continue.
3. i=3: 3/2=1, continue.
4. i=4: $4/2=2 \neq 1$, sum=1+4=5.
5. i=5: $5/2=2 \neq 1$, sum=5+5=10.

9.4 Ausgabe

Ausgabe: 10

9.5 Zusätzliche Erklärungen

`continue` überspringt i=2,3. Summe: 1+4+5=10.

10 Frage 8

10.1 Code

```java
int x = 3, y = 4;
if (x++ / 2 >= y--) {
  System.out.print("A");
} else {
  System.out.print("B");
}
System.out.print(x + y);
```

10.2 Initialisierung

x=3, y=4.

10.3 Schritt-für-Schritt-Ausführung

1. x++=3, x=4, y--=4, y=3, $3/2=1 \geq 4 \rightarrow$ falsch, print "B".
2. x+y=4+3=7, print 7.

10.4 Ausgabe

Ausgabe: B7

10.5 Zusätzliche Erklärungen

Post-Inkrement/Dekrement in der Bedingung verwenden aktuelle Werte, ändern `x`, `y` danach.

11 Frage 9

11.1 Code

```java
int count = 0;
for (int i = 0; i < 4; i++) {
  for (int j = i; j < 4; j++) {
    if (j / 2 == 1) break;
    count++;
  }
}
System.out.print(count);
```

11.2 Initialisierung

count=0, i=0.

11.3 Schritt-für-Schritt-Ausführung

1. i=0: j=0, 0/2=0, count=1; j=1, 1/2=0, count=2; j=2, 2/2=1, break.
2. i=1: j=1, 1/2=0, count=3; j=2, break.
3. i=2: j=2, break.
4. i=3: j=3, break.

11.4 Ausgabe

Ausgabe: 3

11.5 Zusätzliche Erklärungen

Innere Schleife bricht bei `j/2==1` (j=2). `count` zählt Iterationen ohne break.

12 Frage 10

12.1 Code

```
int x = 6;
while (x-- > 3) {
  System.out.print(x / 2);
  if (x == 4) break;
}
```

12.2 Initialisierung

x=6.

12.3 Schritt-für-Schritt-Ausführung

1. x--=6, x=5, 6>3, print 5/2=2.
2. x--=5, x=4, 5>3, print 4/2=2, x==4, break.

12.4 Ausgabe

Ausgabe: 22

12.5 Zusätzliche Erklärungen

`x--` in der Bedingung verwendet den aktuellen Wert. `break` stoppt bei x=4.

13 Frage 11

13.1 Code

```
int x = 3, y = 4;
for (int i = 0; i < 3; i++) {
  x += y++ / 2;
}
System.out.print(x + ":" + y);
```

13.2 Initialisierung

x=3, y=4, i=0.

13.3 Schritt-für-Schritt-Ausführung

1. i=0: y++=4, y=5, 4/2=2, x=3+2=5.
2. i=1: y++=5, y=6, 5/2=2, x=5+2=7.
3. i=2: y++=6, y=7, 6/2=3, x=7+3=10.
4. print 10:7.

13.4 Ausgabe

Ausgabe: 10:7

13.5 Zusätzliche Erklärungen

`y++` erhöht `y` nach der Berechnung. Integer-Division rundet ab.

14 Frage 12

14.1 Code

```
for (int i = 1; i <= 3; i++) {
  for (int j = 3; j >= i; j--) {
    System.out.print(j / 2);
  }
  System.out.println();
}
```

14.2 Initialisierung

i=1.

14.3 Schritt-für-Schritt-Ausführung

1. i=1: j=3, print 1; j=2, print 1; j=1, print 0; `println()`.
2. i=2: j=3, print 1; j=2, print 1; `println()`.
3. i=3: j=3, print 1; `println()`.

14.4 Ausgabe

Ausgabe:

```
110
11
1
```

14.5 Zusätzliche Erklärungen

Innere Schleife läuft rückwärts von j=3 bis j≥i. `println()` fügt Zeilenumbrüche hinzu.

15 Frage 13

15.1 Code

```
int x = 4;
int y = (--x / 2 == 1) ? x++ : ++x;
System.out.print(y);
```

15.2 Initialisierung

x=4.

15.3 Schritt-für-Schritt-Ausführung

1. --x=3, x=3, 3/2=1, wahr, y=x++=3, x=4.
2. print y=3.

15.4 Ausgabe

Ausgabe: 3

15.5 Zusätzliche Erklärungen

Pre-Dekrement `--x` ändert `x` vor der Berechnung. Ternärer Operator weist `y` den Wert von `x++` zu.

16 Frage 14

16.1 Code

```
int i = 0;
while (i++ < 5) {
  if (i / 2 == 2) continue;
  System.out.print(i);
}
```

16.2 Initialisierung

i=0.

16.3 Schritt-für-Schritt-Ausführung

1. i++=0, i=1, print 1.
2. i++=1, i=2, print 2.
3. i++=2, i=3, print 3.
4. i++=3, i=4, 4/2=2, continue.
5. i++=4, i=5, print 5.

16.4 Ausgabe

Ausgabe: 1235

16.5 Zusätzliche Erklärungen

`continue` überspringt i=4. `i++` in der Bedingung inkrementiert vor der Prüfung.

17 Frage 15

17.1 Code

```
int x = 5;
do {
  System.out.print(x / 2);
  x--;
} while (x > 2 && x / 2 == 1);
```

17.2 Initialisierung

x=5.

17.3 Schritt-für-Schritt-Ausführung

1. x=5, print 5/2=2, x=4.
2. $x = 4$, 4>2 `&&` 4/2=2 $\neq 1 \rightarrow$ falsch, Ende.

17.4 Ausgabe

Ausgabe: 2

17.5 Zusätzliche Erklärungen

Do-while-Schleife führt einmal aus. Logische UND-Bedingung (`&&`) schlägt bei x=4 fehl.

18 Frage 16

18.1 Code

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
  for (int j = 1; j <= i; j++) {
    if (j / 2 == 1) sum += j;
  }
}
System.out.print(sum);
```

18.2 Initialisierung

sum=0, i=1.

18.3 Schritt-für-Schritt-Ausführung

1. i=1: j=1, $0 \neq 1$.
2. i=2: j=1; j=2, 2/2=1, sum=2.
3. i=3: j=1; j=2, sum=4; j=3, 3/2=1, sum=7.
4. i=4: j=1; j=2, sum=9; j=3, sum=12; j=4, $2 \neq 1$.

18.4 Ausgabe

Ausgabe: 12

18.5 Zusätzliche Erklärungen

Summiert `j`-Werte, bei denen `j/2==1` (j=2,3). Innere Schleife wächst mit `i`.

19 Frage 17

19.1 Code

```
int x = 3;
if (x++ / 2 == 1 && ++x / 2 == 2) {
  System.out.print("A");
} else {
  System.out.print("B");
}
System.out.print(x);
```

19.2 Initialisierung

x=3.

19.3 Schritt-für-Schritt-Ausführung

1. x++=3, x=4, 3/2=1, wahr.
2. ++x, x=5, 5/2=2, wahr, print Ä".
3. print x=5.

19.4 Ausgabe

Ausgabe: A5

19.5 Zusätzliche Erklärungen

Logische UND-Bedingung (`&&`) wertet beide Teile aus, da die erste Bedingung wahr ist.

20 Frage 18

20.1 Code

```
int x = 6;
for (int i = 1; i <= 3; i++) {
  x -= i / 2;
}
System.out.print(x);
```

20.2 Initialisierung

x=6, i=1.

20.3 Schritt-für-Schritt-Ausführung

1. i=1: 1/2=0, x=6.
2. i=2: 2/2=1, x=5.
3. i=3: 3/2=1, x=4.

20.4 Ausgabe

Ausgabe: 4

20.5 Zusätzliche Erklärungen

Integer-Division reduziert `x` basierend auf `i/2`.

21 Frage 19

21.1 Code

```
for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 3; j++) {
    if (i + j / 2 == 2) break;
    System.out.print(j);
  }
}
```

21.2 Initialisierung

i=0.

21.3 Schritt-für-Schritt-Ausführung

1. i=0: j=0,1,2 → print 012.
2. i=1: j=0,1, 1+1=2, break → print 01.
3. i=2: j=0, 2+0=2, break → print nichts.

21.4 Ausgabe

Ausgabe: 01201

21.5 Zusätzliche Erklärungen

`break` in der inneren Schleife stoppt bei `i+j/2==2`.

22 Frage 20

22.1 Code

```
int x = 4;
while (--x > 1) {
  System.out.print(x / 2);
  if (x == 2) break;
}
```

22.2 Initialisierung

x=4.

22.3 Schritt-für-Schritt-Ausführung

1. --x=3, 3>1, print 3/2=1.
2. --x=2, 2>1, print 2/2=1, x==2, break.

22.4 Ausgabe

Ausgabe: 11

22.5 Zusätzliche Erklärungen

Pre-Dekrement `--x` ändert `x` vor der Bedingung. `break` stoppt bei x=2.

23 Frage 21

23.1 Code

```
int x = 3, y = 4;
for (int i = 0; i < 2; i++) {
  y += x++ / 2;
}
System.out.print(x + ":" + y);
```

23.2 Initialisierung

x=3, y=4, i=0.

23.3 Schritt-für-Schritt-Ausführung

1. i=0: x++=3, x=4, 3/2=1, y=4+1=5.
2. i=1: x++=4, x=5, 4/2=2, y=5+2=7.
3. print 5:7.

23.4 Ausgabe

Ausgabe: 5:7

23.5 Zusätzliche Erklärungen

`x++` erhöht `x` nach der Berechnung. `y` akkumuliert die Ergebnisse.

24 Frage 22

24.1 Code

```java
for (int i = 1; i <= 3; i++) {
  for (int j = i; j >= 1; j--) {
    System.out.print(j / 2);
  }
  System.out.println();
}
```

24.2 Initialisierung

i=1.

24.3 Schritt-für-Schritt-Ausführung

1. i=1: j=1, print 0, `println()`.
2. i=2: j=2, print 1; j=1, print 0, `println()`.
3. i=3: j=3, print 1; j=2, print 1; j=1, print 0, `println()`.

24.4 Ausgabe

Ausgabe:

```
0
10
110
```

24.5 Zusätzliche Erklärungen

Innere Schleife läuft rückwärts von `j=i` bis 1. Integer-Division erzeugt 0 oder 1.

25 Frage 23

25.1 Code

```java
int x = 5;
int y = (x-- / 2 > 2) ? x++ : --x;
System.out.print(y);
```

25.2 Initialisierung

x=5.

25.3 Schritt-für-Schritt-Ausführung

1. x--=5, x=4, 5/2=2 > 2 → falsch, y=--x=3, x=3.
2. print y=3.

25.4 Ausgabe

Ausgabe: 3

25.5 Zusätzliche Erklärungen

Ternärer Operator: `x--` verwendet 5, dann `x=4`. Falsche Bedingung führt zu `--x`.

26 Frage 24

26.1 Code

```
int i = 0;
while (i++ < 4) {
  if (i / 2 == 1) continue;
  System.out.print(i);
}
```

26.2 Initialisierung

i=0.

26.3 Schritt-für-Schritt-Ausführung

1. i++=0, i=1, print 1.
2. i++=1, i=2, 2/2=1, continue.
3. i++=2, i=3, print 3.
4. i++=3, i=4, 4<4 → falsch.

26.4 Ausgabe

Ausgabe: 13

26.5 Zusätzliche Erklärungen

`continue` überspringt i=2. Ähnlich wie Frage 14, aber kürzere Ausgabe.

27 Frage 25

27.1 Code

```
int x = 6;
do {
  System.out.print(x / 2);
  x--;
} while (x > 3 || x / 2 == 2);
```

27.2 Initialisierung

x=6.

27.3 Schritt-für-Schritt-Ausführung

1. x=6, print 6/2=3, x=5, 5>3 → wahr.
2. x=5, print 5/2=2, x=4, 4>3 → wahr.
3. x=4, print 4/2=2, x=3, 3>3 || 3/2=1 ≠ 2 → falsch.

27.4 Ausgabe

Ausgabe: 322

27.5 Zusätzliche Erklärungen

Logische ODER-Bedingung (`||`) hält die Schleife länger aktiv als Frage 15.

28 Frage 26

28.1 Code

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
  if (i / 2 == 2) sum += i;
}
System.out.print(sum);
```

28.2 Initialisierung

sum=0, i=1.

28.3 Schritt-für-Schritt-Ausführung

1. i=1,2,3: i/2 ≠ 2, skip.
2. i=4: 4/2=2, sum=4.
3. i=5: 5/2=2, sum=9.

28.4 Ausgabe

Ausgabe: 9

28.5 Zusätzliche Erklärungen

Summiert `i`-Werte, bei denen `i/2==2` (i=4,5).

29 Frage 27

29.1 Code

```
int x = 2;
if (++x / 2 == 1 || x++ / 2 == 1) {
  System.out.print("A");
} else {
  System.out.print("B");
}
System.out.print(x);
```

29.2 Initialisierung

x=2.

29.3 Schritt-für-Schritt-Ausführung

1. ++x, x=3, 3/2=1, wahr → Kurzschluss, `x++` wird nicht ausgewertet.
2. print Ä", x=3.

29.4 Ausgabe

Ausgabe: A3

29.5 Zusätzliche Erklärungen

Logische ODER-Bedingung (`||`) führt zu Kurzschluss, wenn die erste Bedingung wahr ist.

30 Frage 28

30.1 Code

```
int count = 0;
for (int i = 0; i < 3; i++) {
  for (int j = 0; j <= i; j++) {
    count += j / 2;
  }
}
System.out.print(count);
```

30.2 Initialisierung

count=0, i=0.

30.3 Schritt-für-Schritt-Ausführung

1. i=0: j=0, 0/2=0.
2. i=1: j=0,1, 0+0=0.
3. i=2: j=0,1,2, 0+0+1=1, count=1.

30.4 Ausgabe

Ausgabe: 1

30.5 Zusätzliche Erklärungen

`count` summiert `j/2`. Nur j=2 trägt (1) bei.

31 Frage 29

31.1 Code

```
int x = 2, y = 5;
for (int i = 0; i < 2; i++) {
  x += ++y / 2;
}
System.out.print(x + ":" + y);
```

31.2 Initialisierung

x=2, y=5, i=0.

31.3 Schritt-für-Schritt-Ausführung

1. i=0: ++y=6, 6/2=3, x=2+3=5.
2. i=1: ++y=7, 7/2=3, x=5+3=8.
3. print 8:7.

31.4 Ausgabe

Ausgabe: 8:7

31.5 Zusätzliche Erklärungen

Pre-Inkrement `++y` erhöht `y` vor der Berechnung.

32 Frage 30

32.1 Code

```
int x = 5;
int y = (--x / 2 == 2) ? x++ : ++x;
System.out.print(y);
```

32.2 Initialisierung

x=5.

32.3 Schritt-für-Schritt-Ausführung

1. --x=4, x=4, 4/2=2, wahr, y=x++=4, x=5.
2. print y=4.

32.4 Ausgabe

Ausgabe: 4

32.5 Zusätzliche Erklärungen

Pre-Dekrement `--x` ändert `x` vor der Bedingung. `x++` weist `y` den aktuellen Wert zu.